

05

1시간

Skill과 Subagent

커스텀 명령

Agent Skills

Subagent

MCP 서버

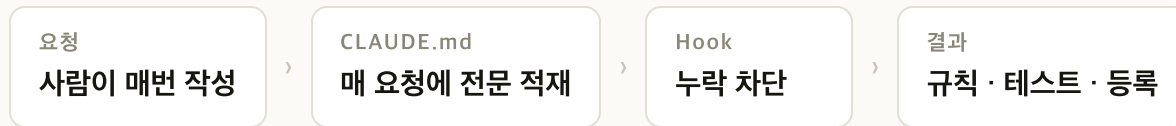
규칙 일곱 개

이 챕터에서 다루는 내용

주제	내용	실습
커스텀 명령	이름을 붙여 저장한 요청 문장	1
Agent Skills	부를 때만 컨텍스트에 오르는 절차	2
규칙 일곱 개	R011~R016과 R018	3
Subagent	별도 컨텍스트에서 규정과 구현을 대조	4
MCP 서버	내장 도구가 닿지 않는 곳을 잇는 통로	시연

챕터 4에서는 절차를 파일에 적었고, 이 챕터에서는 그 절차를 실행하는 일까지 넘깁니다.
규칙 일곱 개도 직접 작성하지 않고 여기서 만든 도구로 처리합니다.

챕터 4에서 남은 과제



- **사람에게 남은 일:** 규약을 파일에 적어 두었어도 무엇을 할지 지시하는 문장은 매번 직접 써야 합니다. 규칙 일곱 개를 만들려면 같은 형태의 요청을 일곱 번 쓰게 됩니다.
- **매 요청에 실리는 것:** CLAUDE.md 의 규칙 추가 절차와 @ 로 붙인 경비규정 전문은 규칙을 만들 때만 필요합니다. 테스트 실패 원인을 묻는 요청에도, 출력 표에 열을 추가하는 요청에도 절차와 규정 스무 항목이 함께 올라갑니다.

되풀이하는 것에는 이름을 붙여 따로 둡니다.

요청 문장은 커스텀 명령으로, 절차는 Skill로 옮깁니다.

커스텀 명령의 구조

.claude/commands/rule-status.md · `/rule-status` 로 호출

```

---
description: 아직 구현하지 않은 규칙 코드를 확인합니다
allowed-tools: Bash(grep:*), Bash(ls:*)
---
# 아래 본문이 그대로 요청 문장이 됩니다.
규정에 실린 규칙: !`grep "^### R" docs/경비규정.md`
구현된 규칙 파일: !`ls src/core/domain/rules`
두 목록을 대조해 아직 구현하지 않은 규칙만 알려 주세요. ...

```

- **저장 위치:** 프로젝트는 `.claude/commands/`, 사용자 전역은 `~/.claude/commands/` 입니다.
- **인자 받기:** 본문에 `$ARGUMENTS` 를 적으면 명령 뒤에 붙인 문자열이 그 자리에 들어갑니다. `$0` `$1` 처럼 번호를 붙이면 인자를 순서대로 하나씩 받습니다. 번호는 0부터 시작합니다.

동적 컨텍스트 주입

	!`명령`	@경로	Hook
들어가는 것	명령의 실행 결과	파일 내용	스크립트가 출력한 문자열
쓸 수 있는 곳	커스텀 명령, Skill	CLAUDE.md, 커스텀 명령, Skill	설정 파일의 이벤트
들어가는 시점	명령을 부를 때	그 파일이 적재될 때	이벤트가 발생할 때
CLAUDE.md에서	실행되지 않음	내용이 그대로 실림	해당 없음

!`명령` 은 커스텀 명령과 Skill에서만 실행됩니다.
CLAUDE.md 에 적으면 실행되지 않고 문자 그대로 남습니다.

실습 1 · 규칙 현황 명령 만들기

- 1 **요청:** docs/경비규정.md 의 규칙 코드와 src/core/domain/rules/ 의 파일을 대조해 아직 구현하지 않은 코드를 알려 주는 커스텀 명령을 /rule-status 로 만들어 줘. 두 목록은 셸 명령의 실행 결과로 넣어 줘
- 2 .claude/commands/rule-status.md 를 열어 본문에 !`grep ...` 과 !`ls ...` 가 들어갔는지 확인
- 3 /rule-status 실행
- 4 **요청:** 규칙 코드를 인자로 받아 해당 규정 항목을 설명하는 명령을 /rule-spec 으로 만들어 줘
- 5 /rule-spec R013 실행. 인자가 본문의 어느 자리에 들어갔는지 파일에서 확인

실행 결과가 요청 문장에 먼저 들어갑니다.

모델이 도구를 쓰지 판단하는 단계를 거치지 않으므로 명령을 실행할 때마다 같은 자료가 실립니다.

/rule-status 실행 결과

PowerShell · /rule-status

```
R011 · 접대비 참석인원 누락 · 단건
R012 · 접대비 1인 한도 · 단건
R013 · 주말 사용 사전승인 · 단건
R014 · 적격증빙 누락 · 단건
R015 · 계정과목·업종 불일치 · 단건
R016 · 개인카드 사용 사유 누락 · 단건
R017 · 중복 청구 · 다건
R018 · 현금 결제 한도 · 단건
R019 · 부서 월 한도 초과 · 다건
R020 · 1일 식비 중복 · 다건
```

이 목록에서 단건 규칙 일곱 개를 골라 실습 3에서 만듭니다.

/rule-spec 실행 결과

PowerShell · /rule-spec R013

R013 · 주말 사용 사전승인 (단건)

- 규칙 유형: 단건 (청구 한 건만으로 판정)
- 판정 조건: 사용일자가 토요일 또는 일요일인데 비고에 `A-` 로 시작하는 사전승인 번호가 없으면 반려
- 위반 문구: 주말 사용 건은 사전승인 번호가 필요합니다

- 인자가 들어가는 자리: 본문의 \$ARGUMENTS 가 모두 R013 으로 바뀝니다. 규정 문서에서 해당 항목을 찾는 셀 명령 안에도 들어갑니다.
- 규정 적재 범위: 항목 하나만 뽑아 넣으므로 스무 항목을 통째로 올리지 않습니다.

Skill을 두는 두 가지 형태

	.claude/commands/이름.md	.claude/skills/이름/SKILL.md
단위	파일 하나	폴더 하나
만들어지는 명령		/이름
인자와 자동 호출		\$ARGUMENTS · 설명을 보고 모델이 판단
함께 두는 파일	둘 수 없음	스크립트, 참조 문서를 둘 수 있음
호출 주체 제어	제한 없음: 사용자와 모델 모두 호출	사용자만 호출: disable-model-invocation: true 모델만 호출: user-invocable: false

커스텀 명령은 Skill에 합쳐졌습니다.

어느 쪽에 두어도 같은 /이름 이 만들어집니다. 함께 둘 파일이 생기거나, 호출 주체를 한쪽으로 제한해야 할 때 폴더 형태로 옮깁니다.

Skill의 구조와 적재 방식

.claude/skills/add-rule/ · 폴더 이름이 명령 이름

```
add-rule/
├── SKILL.md           절차와 규약
└── scripts/check_rule.py 본문이 부르는 스크립트
# SKILL.md
name: add-rule
description: 규칙 코드를 받아 구현, 테스트, 등록까지 처리합니다.
             규칙을 추가할 때 사용합니다.
```

- **적재되는 시점:** 평소에는 `name` 과 `description` 만 올라가 있습니다. 본문은 부를 때 실리고, 한 번 실리면 그 세션 동안 남습니다.
- **`description` 의 역할:** 모델은 이 한 줄만 보고 Skill을 부를지 정합니다. 무엇을 하는지와 언제 쓰는지를 함께 적습니다.
- **함께 두는 파일:** 스크립트와 참조 문서를 폴더에 같이 넣고 본문에서 `${CLAUDE_SKILL_DIR}` 로 가리킵니다. 같은 경로를 `allowed-tools` 에도 적어 두면 실행할 때 승인을 묻지 않습니다.

실습 2 · 규칙 추가 Skill 만들기

- 1 `/context` 로 현재 프로젝트 메모리 적재량 확인
- 2 **요청:** `CLAUDE.md` 의 규칙 추가 절차를 `add-rule` Skill로 옮겨 줘. 규칙 코드를 받아 구현, 테스트, 등록까지 하고 규정 문서도 Skill 안에서 참조하게 해 줘
- 3 **요청:** 규칙 구현 파일, 테스트, 픽스처, 앱 컨테이너 등록, 테스트 컨테이너 등록 다섯 곳에 규칙 하나가 모두 반영됐는지 확인하는 스크립트를 `scripts/check_rule.py` 로 Skill 폴더에 만들고, 절차에서 `${CLAUDE_SKILL_DIR}` 경로로 호출하게 하고 `allowed-tools` 에도 등록해 줘
- 4 `SKILL.md` 를 열어 절차의 실행 명령이 `uv run` 인지 확인. `${CLAUDE_SKILL_DIR}` 를 실제 경로로 바꾸지 말라는 지시 문장이 없으면 직접 추가
- 5 `uv run python .claude/skills/add-rule/scripts/check_rule.py R010` 실행. 이미 만든 규칙으로 동작 확인
- 6 `/context` 를 다시 실행해 Skills 항목에 `add-rule` 이 올라왔는지 확인

절차가 사라진 것이 아니라 부를 때만 컨텍스트에 오르도록 자리를 옮긴 것입니다.
규칙을 만들지 않는 요청에서는 이제 이 절차와 경비규정 전문이 올라가지 않습니다.

실습 2 결과 파일

SKILL.md · 스크립트를 부르는 자리

```
allowed-tools: Bash(uv run python ${CLAUDE_SKILL_DIR}/scripts/check_rule.py:*)
...
5. `uv run python ${CLAUDE_SKILL_DIR}/scripts/check_rule.py <규칙 코드>` 로 연동 지점
   다섯 곳이 모두 반영됐는지 확인한다.
   ${CLAUDE_SKILL_DIR} 를 실제 경로로 바꾸지 말고 위에 적힌 그대로 실행한다.
6. `uv run pytest` 로 전체가 통과하는지 확인한다.
```

매 요청에 실리는 것	챕터 4를 마쳤을 때	지금
CLAUDE.md	규칙 추가 절차를 포함한 전문	절차를 뺀 나머지만 실림
r001_required_fields.py	실림	빠짐
docs/경비규정.md	전문이 실림	빠짐
add-rule 이름과 설명	없음	한 줄만 실림

실습 3 · 규칙 일곱 개

- 1 `/rule-status` 로 아직 만들지 않은 규칙 확인
- 2 `/add-rule R011 R012 R013 R014 R015 R016 R018` 실행
- 3 규칙마다 `check_rule.py` 가 승인 프롬프트 없이 실행되어 연동 지점 다섯 곳을 점검하는지 화면에서 확인
- 4 골든 세트 set-2 를 내려받아 `tests\golden\set-2` 에 배치
- 5 `uv run pytest` 로 `19 passed` 확인 (골든 세트 두 개)
- 6 커밋. 챕터 4에서 만든 Hook이 등록을 검사하고 통과시키는지 확인

이름을 쓰지 않고 요청해도 모델이 `description` 을 보고 Skill을 고를 수 있습니다.
다만 고르지 않을 때도 있으므로 여기서는 `/add-rule` 로 직접 부릅니다.

실습 3 실행 결과

Skill이 규칙마다 실행한 check_rule.py

R011 연동 지점 점검

[OK] 규칙 구현: r011_entertainment_attendees.py

[OK] 테스트: test_r011.py

[OK] 픽스처: r011.csv

[OK] 앱 조립: import 와 목록 등록 확인

[OK] 테스트 조립: import 와 목록 등록 확인

다섯 곳 모두 반영됐습니다.

규칙 일곱 개를 모두 만들고 골든 세트를 배치하면 `uv run pytest` 결과는 `19 passed` 가 됩니다.

`/rule-status` 를 다시 실행하면 R017 · R019 · R020 만 남습니다. 세 규칙 모두 다건 규칙이므로 챕터 7에서 다룹니다.

컨텍스트를 나누는 Subagent

- **Subagent:** 시스템 프롬프트와 도구 목록을 따로 지정하는 Agent입니다. 자체 컨텍스트 창에서 실행되고 마지막 응답만 돌려줍니다.
- **따로 맡기는 일:** 파일을 여러 개 읽어야 하는 조사, 같은 대상을 다른 시선으로 보는 검토, 권한을 제한해야 하는 실행입니다. 중간 과정이 길고 결과만 필요하다는 공통점이 있습니다.
- **이 챕터에서 맡길 일:** 규정과 구현의 대조입니다. 규칙 일곱 개를 만든 대화에서 그대로 이어 검토하면 모델은 규정이 아니라 앞서 스스로 내린 판단을 다시 읽습니다. 그래서 잘못된 결과물도 맞다고 판단하고 넘어가기 쉽습니다.

Subagent는 이 대화와 분리된 자리에서 실행됩니다.

컨텍스트: 지금까지의 대화가 넘어가지 않고 중간 과정도 돌아오지 않습니다.

지시: 전달한 내용만 수행합니다.

권한: 허용한 도구 외에는 사용할 수 없습니다.

Subagent의 구조

.claude/agents/rule-spec-check.md

```

---
name: rule-spec-check
description: 경비규정 문서와 규칙 구현을 대조합니다. 규칙을 추가한 뒤 사용합니다.
tools: Read, Grep, Glob
---

# 아래 본문이 Subagent의 시스템 프롬프트가 됩니다.
당신은 규정 문서와 구현 코드를 대조하는 검토자입니다.

...

```

- **tools** : 허용할 도구만 나열합니다. **Edit** 와 **Write** 를 빼면 코드를 고치지 못하고 보고만 합니다.
- **돌아오는 것**: 마지막 응답 하나입니다. 무엇을 읽었고 어떻게 판단했는지는 넘어오지 않습니다.

세 장치의 컨텍스트 비교

	CLAUDE.md	Skill	Subagent
담는 것	프로젝트 전반의 기본 지침	되풀이하는 업무 절차	따로 맡길 작업
컨텍스트	매 요청에 함께 실림	부를 때 실리고 세션 동안 남음	별도 창에서 실행
부르는 방법	자동	/이름 또는 모델이 판단	이름 지정 또는 모델이 판단
남는 것	대화에 그대로 쌓임		마지막 응답만 남음

컨텍스트를 나누는 장치는 Subagent뿐입니다.
 앞의 둘은 실리는 시점이 다를 뿐 모두 같은 대화에 쌓입니다.

실습 4 · 명세 대조 Subagent

- 1 **요청:** docs/경비규정.md 와 규칙 구현을 대조하는 Subagent를 rule-spec-check 로 만들어 줘. 읽기 도구만 주고 코드는 고치지 못하게 해. 규칙마다 code 와 name , 판정 조건, 위반 문구, 상속 클래스를 대조하고, 다른 점이 있는 규칙만 보고하게 해 줘
- 2 .claude/agents/rule-spec-check.md 의 tools 에 Edit 와 Write 가 있는지 확인
- 3 **세션 재시작:** Claude Code를 종료하고 다시 실행
- 4 **호출:** rule-spec-check Subagent 로 R011부터 R018까지 규정과 구현을 대조해 줘
- 5 실행 중에 ↓ 를 눌러 백그라운드 목록을 열고 rule-spec-check 를 선택해 진행 상황 확인
- 6 보고받은 항목을 규정 문서에서 직접 확인

세션 도중에 만든 Subagent는 그 세션에서 호출하지 못합니다.

Claude Code는 커스텀 Agent 목록을 세션이 열릴 때 한 번 읽고 중간에 다시 읽지 않습니다. 파일을 만든 뒤 Claude Code를 다시 실행해야 목록에 올라옵니다.

Subagent의 백그라운드 실행 화면

```

> rule-spec-check Subagent 로 R011부터 R018까지 규정과 구현을 대조해 줘

● rule-spec-check(R011~R018 규정-구현 대조)
  | Backgrounded agent (↓ to manage · ctrl+o to expand)

● R011~R018 규정-구현 대조를 rule-spec-check 서브에이전트에 맡겼어. 백그라운드로 실행 중이고, 끝나면 결과를 알려줄게.

* Waiting for 1 background agent to finish

> 결과 나오면 알려줘

↑/↓ to select · Enter to view

> ● main
  ○ rule-spec-check Reading r013.csv fixture and test_r013.py
    
```

목록에서 선택한 rule-spec-check 의 진행 상황 표시

- **본 대화 유지:** Subagent가 실행되는 동안에도 입력할 수 있습니다. 화면 아래에 실행 중인 개수가 표시됩니다.
- **진행 상황 확인 방법:** ↓ 로 목록을 열어 선택하고, ctrl+o 로 펼칩니다. 별도 컨텍스트에서 일어나는 일이라 본 대화에는 쌓이지 않습니다.

실습 4 대조 결과

rule-spec-check · R011부터 R018까지 대조

R011~R016, R018 차이 없음

code · name, 판정 조건, 위반 문구, 상속 클래스가 규정과 같습니다

R017 미구현

규정: 사번 · 사용일자 · 금액 · 가맹점이 모두 같은 건이 2건 이상

구현: 규칙 파일이 없고 두 컨테이너 모두 R016 다음이 R018 입니다

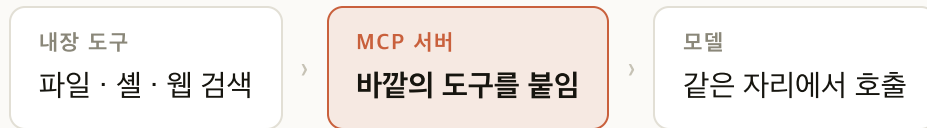
참고

R014 와 R018 은 정확히 50,000원인 경계값이 픽스처에 없습니다

구현이 없는 규칙도 대조 결과에 올라옵니다.

R017은 다건 규칙이므로 챕터 7에서 다룹니다.

내장 도구의 경계



- **내장 도구가 닿는 곳:** 파일 읽기와 쓰기, 셀 실행, 웹 검색입니다. 저장소 안에서 답을 찾는 일은 여기까지입니다.
- **달지 않는 곳:** 지금 쓰는 라이브러리 버전의 문서, 사내 이슈 트래커, 운영 데이터베이스처럼 저장소 밖에 있고 검색만으로는 정확도가 떨어지는 영역입니다.
- **MCP:** 그런 곳을 도구로 잇는 규약입니다. 서버를 붙이고 나면 내장 도구와 같은 방식으로 호출됩니다.

MCP 서버 연결과 범위

PowerShell · 원격 서버를 프로젝트 범위로 등록

```

claude mcp add --transport http context7 https://mcp.context7.com/mcp --scope project

```

범위	저장되는 곳	적용 대상
local	내 설정 파일 (기본값)	이 프로젝트에서 나만
user	내 설정 파일	모든 프로젝트에서 나만
project	<code>.mcp.json</code>	저장소를 공유하는 사람 모두

등록된 서버는 `/mcp` 로 확인합니다.
 연결 상태와 그 서버가 제공하는 도구 목록이 함께 표시됩니다.

강사 시연 · Context7

요청 한 번에 차례로 호출되는 서버의 도구 두 개

```
# 요청
FastAPI의 BackgroundTasks 사용법을 Context7로 찾아 줘

# 1. resolve-library-id · 라이브러리 이름을 문서 ID로 바꿈
- Context7-compatible library ID: /websites/fastapi_tiangolo
- Code Snippets: 2607 · Source Reputation: High
여러 후보 가운데 모델이 하나를 고름

# 2. query-docs · 고른 ID로 문서와 예제를 받음
### Schedule background tasks in FastAPI route
Source: https://fastapi.tiangolo.com/reference/background
background_tasks.add_task(write_notification, email, message="...")
```

이 항목은 강사 시연으로 진행합니다.

실습 PC의 네트워크 정책에 따라 외부 서버 접속이 막힐 수 있습니다. 동작은 화면으로 확인하고, 연결 절차는 앞 슬라이드의 명령을 참고해 주세요.

개발 주변 업무의 MCP 연결

서버	있는 곳	요청 예시
Atlassian	Jira	JIRA-1234 를 읽고 재현 테스트를 만든 다음, 고친 내용을 이슈에 댓글로 남겨 줘
Atlassian	Confluence	설계 문서 페이지와 지금 구현이 다른 곳을 찾아 표로 정리해 줘
Figma	디자인 파일	이 프레임의 색과 간격, 컴포넌트 이름을 읽어 React 컴포넌트로 만들어 줘
pen.dev	로컬 .pen 파일	정산 결과 화면의 표 레이아웃을 만들고 스크린샷으로 확인해 줘
Google Workspace	Drive	팀 드라이브의 요구사항 시트를 읽어 테스트 케이스 목록으로 옮겨 줘
Google Workspace	Gmail	배포 결과와 남은 이슈를 정리해 담당자에게 보낼 메일 초안을 만들어 줘

맥락을 사람이 옮겨 적지 않아도 됩니다.

이슈 내용, 설계 문서, 디자인 값, 요구사항 시트는 모두 저장소 밖에 있습니다. MCP로 이으면 사람이 요약해 붙여 넣지 않아도 Claude Code가 원본을 그대로 읽고, 결과도 이슈 댓글이나 메일 초안처럼 원래 자리에 남깁니다.

하네스에 더한 구성

저장소 루트 · 챕터 5를 마친 상태

CLAUDE.md	기술 스택 · 폴더 구조 · 조립 규칙 · 금지
.mcp.json	MCP 서버, 강사 시연에서 추가
.claude/	
├─ settings.json	환경변수 · 권한 · Hook 등록
├─ hooks/	세션 상태 주입 · 등록 누락 차단
├─ commands/	되풀이하는 요청 두 개
├─ skills/add-rule/	규칙 추가 절차와 점검 스크립트
└─ agents/rule-spec-check.md	규정과 구현 대조

- **넘긴 것:** 챕터 4에서는 규약만 적어 두고 요청은 사람이 했습니다. 이 챕터에서는 요청과 절차까지 파일에 두었습니다.
- **남은 것:** 무엇을 만들지 고르는 일과 결과가 맞는지 판단하는 일은 사람이 합니다.

확인 항목

- ☐ `/rule-status` 커스텀 명령 생성
- ☐ `add-rule` Skill 폴더와 `SKILL.md` 생성
- ☐ `CLAUDE.md` 에서 규칙 추가 절차 제거
- ☐ 본문과 `allowed-tools` 에 같은 `${CLAUDE_SKILL_DIR}` 경로 지정
- ☐ `uv run pytest` 에서 `19 passed` 확인
- ☐ Subagent `tools` 에 읽기 도구만 지정
- ☐ `$ARGUMENTS` 를 쓰는 `/rule-spec` 생성
- ☐ Skill 폴더에 `check_rule.py` 포함
- ☐ 경비규정 참조를 Skill 안으로 이동
- ☐ 규칙 R011~R016, R018 일곱 개 구현
- ☐ 커밋 시 등록 확인 Hook 통과
- ☐ 대조 보고를 규정 문서로 직접 확인